

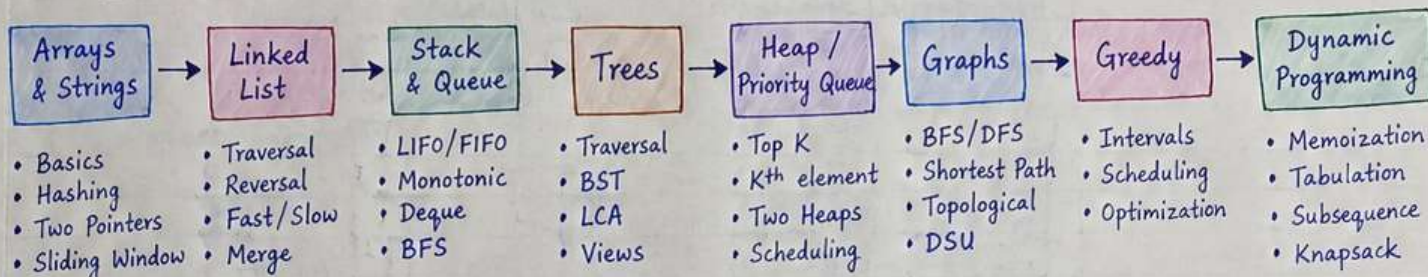
DSA COMPLETE HANDBOOK

Better Problems
Stronger You! 😊

Patterns • Techniques • Problems • Interview Revision

"Same concepts. Different problems. Stronger thinking."

① DSA Roadmap



② Time Complexity Cheat Sheet

Complexity	Name	Example
$O(1)$	Constant	Hash lookup, array access
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Array/Linked list traversal
$O(n \log n)$	Linearithmic	Sorting (Merge/Heap/Quick)
$O(n^2)$	Quadratic	Nested loops
$O(n^3)$	Cubic	3 nested loops
$O(2^n)$	Exponential	Subsets
$O(n!)$	Factorial	Permutations

③ Space Complexity

- Auxiliary space used by the algorithm
- Important in recursion, DP, graph etc.

Common Examples:

- $O(1)$ → In-place operations
- $O(n)$ → Extra array, hash map, recursion stack
- $O(n^2)$ → DP table (2D), adjacency matrix

④ How to Approach a DSA Problem?

- Understand the problem
 - Read carefully, identify input, output, constraints
- Think of examples
 - Try dry run with small cases
- Start with brute force
 - Get a working solution
- Find the pattern
 - Look for known techniques
- Optimize
 - Improve time/space complexity
- Code & test
 - Handle edge cases
- Follow up
 - Can it be further optimized?

⑤ Important Mindset

- Practice regularly (consistency > cramming)
- Learn patterns, not just solutions
- Solve problems on your own before looking up
- Analyze time & space complexity
- Write clean and modular code
- Revise and maintain notes
- Give mock interviews/contests

"It's not just about solving problems, it's about thinking better."



⑥ Key Resources (for practice)

- LeetCode (Must)
- Codeforces (For contests)
- Striver A2Z DSA Sheet (Structured)
- NeetCode (Patterns + Visual)
- GeeksforGeeks (Theory + Practice)

"Solve Learn Repeat Grow" 😊



Discipline today
Dream career tomorrow!

② Arrays & Hashing

"Arrays give you the elements, Hashing gives you the power to find them."

① Why Arrays?

- Store data in contiguous memory.
- Random access in $O(1)$.
- Useful for traversal, searching, sorting.
- Many problems can be solved with simple iteration or with prefix sums / two pointers.

Array representation:

index →	0	1	2	3	4	5
arr →	5	2	8	1	9	3

Access:
arr[i]
 $O(1)$

② Why Hashing?

- Use HashMap/HashSet to store key → value pairs.
- Helps in fast lookups (average $O(1)$).
- Useful for frequency counting, duplicate checks, finding pairs, grouping, etc.
- Turn many $O(n^2)$ problems into $O(n)$.

HashMap example (frequency count):

```
unordered_map<int, int> freq;
for(int x : arr){
    freq[x]++;
}
// freq[x] gives count of x
```

HashSet:

- stores only keys
- useful for existence check
- e.g. unordered_set<int> s;

③ Common Patterns

- 1 Frequency Counting (using HashMap)
- 2 Check for duplicates (using HashSet)
- 3 Prefix Sum / Cumulative Sum
- 4 Subarray Sum (using HashMap)
- 5 Kadane's Algorithm (Maximum Subarray)
- 6 Sorting + Hashing (for pairs/triplets)
- 7 In-place modification (two pointers)

④ Prefix Sum

- Useful for range sum queries.
- prefix[i] = sum of elements from 0 to i-1

Example:

```
arr = [2, 4, 1, 6, 3]
prefix = [0, 2, 6, 7, 13, 16]
```

Time: $O(n)$, Space: $O(n)$

Range sum
from l to r:
= prefix[r+1] - prefix[l]

⑤ Kadane's Algorithm (Max Subarray Sum)

```
int maxSum = INT_MIN, curr = 0;
for(int x : arr){
    curr = max(x, curr + x);
    maxSum = max(maxSum, curr);
}
```

Works because
either start new
subarray or
extend previous one.

Time: $O(n)$
Space: $O(1)$

⑥ Must Know Problems

#	Problem	Pattern / Idea	Difficulty
1	Two Sum	HashMap	Easy
2	Contains Duplicate	HashSet	Easy
3	Maximum Subarray	Kadane's Algorithm	Easy
4	Product of Array Except Self	Prefix / Suffix	Medium
5	Subarray Sum Equals K	Prefix Sum + HashMap	Medium
6	Longest Consecutive Sequence	HashSet	Medium
7	Group Anagrams	HashMap (string hash)	Medium
8	Top K Frequent Elements	HashMap + Heap	Medium
9	3Sum	Sorting + Two Pointers	Medium
10	Subarray Sum Divisible by K	Prefix Sum + HashMap	Medium

⑦ Tips & Tricks

- When looking for a pair → think HashMap or Two Pointers (if sorted).
- For subarray problems → try Prefix Sum.
- For maximum/minimum subarray → Kadane's.
- For counting / grouping → HashMap.
- If elements are in range (e.g. $0 \dots n$) → consider frequency array instead of HashMap.

"Find the pattern,
not just the solution."



3 Two Pointers

1 What is Two Pointers?

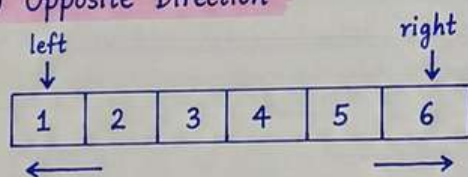
- Use two pointers (indices) to traverse the array/string and solve the problem in $O(n)$ instead of $O(n^2)$.
- Works best on sorted arrays or when problem involves pairs, subarrays or in-place modifications.
- Can be used with opposite direction, same direction or with a gap.

2 Common Patterns

- Opposite direction (start and end)
- Same direction (slow and fast)
- Fixed gap between pointers
- Sort + Two Pointers
- In-place modification (remove/partition)
- Using two pointers on linked list (fast/slow)

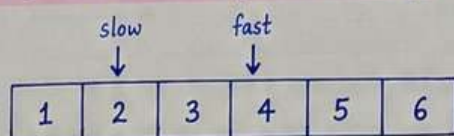
3 Types with Diagrams

(a) Opposite Direction



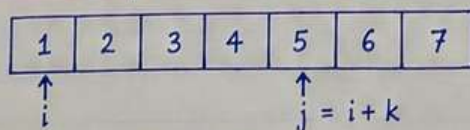
- Move towards each other based on condition.
- Used in pair sum, 3Sum, etc.

(b) Same Direction (Slow & Fast)



- Used in problems like removing duplicates, finding cycles (linked list), subarray problems.

(c) Fixed Gap (Window of size k)



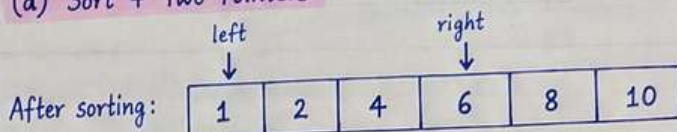
- Maintain a fixed difference between pointers.
- Useful in subarray / window problems.

"Less comparisons,
More progress!"



"Two pointers help you find the answer faster by moving from both ends (or same direction) instead of using nested loops."

(d) Sort + Two Pointers



- Sort the array first, then use two pointers.
- Useful for problems like 3Sum, pair with given sum, etc.

4 Example: Two Sum II (Sorted Array)

Given a sorted array, find two numbers that add up to target.

```
int left = 0, right = n - 1;
while(left < right){
    int sum = arr[left] + arr[right];
    if(sum == target) return {left, right};
    else if(sum < target) left++;
    else right--;
}
return {-1, -1};
```

If sum < target
→ move left++
(bigger number)

If sum > target
→ move right--
(smaller number)

Time: $O(n)$ Space: $O(1)$

5 Example: Remove Duplicates from Sorted Array

```
int i = 0;
for(int j = 1; j < n; j++){
    if(nums[j] != nums[i]){
        i++;
        nums[i] = nums[j];
    }
}
return i + 1; // new length
```

Input:
[1, 1, 2, 2, 3, 3]

Output:
[1, 2, 3]
Length = 3

Time: $O(n)$ Space: $O(1)$

6 Must Solve Problems

#	Problem	Pattern	Difficulty
1	Two Sum II (sorted)	Two Pointers	Easy
2	3Sum	Sort + Two Pointers	Medium
3	Container With Most Water	Opposite Pointers	Medium
4	Trapping Rain Water	Two Pointers	Hard
5	Remove Duplicates from Sorted Array	Two Pointers	Easy
6	Valid Palindrome	Two Pointers	Easy
7	Move Zeroes	Two Pointers (in-place)	Easy
8	Pair with Given Sum (sorted)	Two Pointers	Easy

Think Pattern
Use Two Pointers
Solve Faster!

4 Sliding Window

"Work on a window, not the whole array."

1 What is Sliding Window?

- Technique to solve problems on arrays/strings that involve a subarray / substring (contiguous).
- Instead of recomputing for every subarray, we maintain a window and slide it.
- Helps to optimize brute force ($O(n^2)$) to $O(n)$.

2 Types of Sliding Window

(a) Fixed Size Window

- Window size is constant (k).
- Use when we need to find something in all subarrays of size k .
- Example: Maximum sum of subarray of size k .

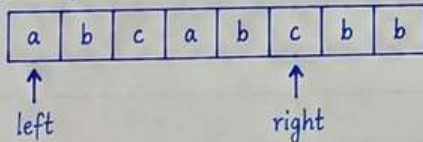
(b) Variable Size Window

- Window size can grow or shrink based on condition.
- Use when we need to find smallest / largest window satisfying a condition.
- Example: Longest substring without repeating characters.

4 Variable Size Window - Example

Problem: Longest Substring Without Repeating Characters

string = "abcabcbb"



Maintain a set of characters in current window.

Pseudo Code:

```

left = 0, maxLen = 0, set = {}
for right in range(n):
    while s[right] in set:
        set.remove(s[left])
        left += 1
    set.add(s[right])
    maxLen = max(maxLen, right - left + 1)
    
```

Time: $O(n)$
Space: $O(\min(n, k))$

We shrink the window when condition is violated.

5 When to use Sliding Window?

- Problem involves contiguous subarray / substring.
- Need to find max/min/length/count under a condition.
- Brute force gives $O(n^2)$, try to maintain a window.
- Common in array, string, and some DP/greedy problems.

"Small window, Big results!"



3 Fixed Size Window - Example

Problem: Maximum Sum Subarray of Size K

arr = [2, 1, 5, 1, 3, 2] $k = 3$

Window slides →

2	1	5	1	3	2	sum = 8
2	1	5	1	3	2	sum = 7
2	1	5	1	3	2	sum = 9 (max)
2	1	1	1	3	2	sum = 6

Pseudo Code:

```

windowSum = 0, maxSum = 0
for i = 0 to k-1: windowSum += arr[i]
maxSum = windowSum
for i = k to n-1:
    windowSum += arr[i] - arr[i-k]
    maxSum = max(maxSum, windowSum)
    
```

Time: $O(n)$
Space: $O(1)$

We add new element and remove old element from window.

6 Common Problems

#	Problem	Type	Difficulty
1	Maximum Sum Subarray of Size K	Fixed	Easy
2	Longest Substring Without Repeating Characters	Variable	Medium
3	Longest Repeating Character Replacement	Variable	Medium
4	Minimum Window Substring	Variable	Hard
5	Permutation in String	Variable	Medium
6	Subarray Sum Equals K (non-negative)	Variable	Medium
7	Fruits Into Baskets	Variable	Medium
8	Max Consecutive Ones III	Variable	Medium

7 Tips & Tricks

- Use a hashmap / set to maintain counts.
- Expand right pointer first, then shrink left if needed.
- Think: what should be inside the window?
- Update answer at every valid step.
- Don't forget edge cases (empty string, $k > n$, etc.).

Slide Smart
Solve Faster

5 Binary Search

"Not faster code,
Just smarter search."

1 What is Binary Search?

- Binary search is used to find an element (or answer) in a sorted array by repeatedly dividing the search space in half.
- Time Complexity: $O(\log n)$
- Works only on sorted data (or monotonic condition).
- At each step, compare with middle element and discard half of the space.

2 Basic Binary Search (Iterative)

0	1	2	3	4	5	6
1	3	5	7	9	11	13
↑		↑				↑
left		mid				right

If target < mid
→ search left
If target > mid
→ search right

Code (Iterative):

```
int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1; // not found
}
```

Use
 $left + (right - left) / 2$
to avoid
integer overflow.

If answer is
monotonic
→ Think
Binary Search
on Answer!

3 Variations

- Find first occurrence (lower bound)
- Find last occurrence (upper bound)
- Find insertion position
- Search in rotated sorted array
- Binary search on answer

4 Lower Bound & Upper Bound

- Lower Bound: first index where $arr[i] \geq target$
- Upper Bound: first index where $arr[i] > target$

Lower Bound:

```
int low = 0, high = n;
while (low < high) {
    int mid = (low + high) / 2;
    if (arr[mid] < target)
        low = mid + 1;
    else
        high = mid;
}
return low;
```

Upper Bound:

```
int low = 0, high = n;
while (low < high) {
    int mid = (low + high) / 2;
    if (arr[mid] <= target)
        low = mid + 1;
    else
        high = mid;
}
return low;
```

5 Binary Search on Answer

- Used when we need to find the minimum/maximum value that satisfies a condition.
- The search space is not the array, but the answer range.
- Condition should be monotonic (if a value works, all greater/smaller values also work).

Steps:

- Define search space (low, high)
- Find mid (possible answer)
- Check if mid is valid using a function
- Adjust search space based on result
- Return the answer

6 Example: Koko Eating Bananas

- Find minimum eating speed (k) to finish all bananas in h hours.
- Search space: $[1, \max(\text{piles})]$
- Check if a given k is valid (can finish in $\leq h$ hours).

```
bool canEat(int[] piles, int k, int h) {
    long hours = 0;
    for (int p : piles) {
        hours += (p + k - 1) / k; // ceil
    }
    return hours <= h;
}
```

If canEat(k)
is true
→ try smaller k
else
→ try larger k

7 Common Problems

#	Problem	Type / Pattern	Difficulty
1	Binary Search	Basic	Easy
2	Search Insert Position	Basic	Easy
3	First & Last Occurrence	Lower/Upper Bound	Medium
4	Search in Rotated Sorted Array	Modified BS	Medium
5	Find Minimum in Rotated Array	Modified BS	Medium
6	Koko Eating Bananas	BS on Answer	Medium
7	Capacity to Ship Packages within D Days	BS on Answer	Medium
8	Median of Two Sorted Arrays	Partition / BS	Hard
9	Find Single Element	Modified BS	Medium
10	Peak Element	Modified BS	Medium

"Shrink the search space,
grow your chances!"

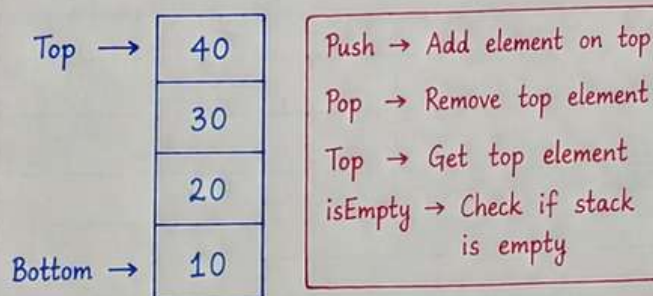


6 Stack

1 What is a Stack?

- A linear data structure that follows LIFO (Last In, First Out) principle.
- Insertion and deletion happen at the top.
- Main operations: push(), pop(), top(), isEmpty()
- Time Complexity: $O(1)$ for all basic operations.

2 Stack Representation



3 Basic Operations (Code)

```
stack<int> st;
st.push(10); // [10]
st.push(20); // [10, 20]
st.push(30); // [10, 20, 30]

st.top(); // 30
st.pop(); // removes 30 → [10, 20]
st.empty(); // false
st.size(); // 2
```

Inbuilt in C++:

```
#include <stack>
stack<int> st;
```

In Java:

```
Stack<Integer> st =
    new Stack<>();
```

4 Common Use Cases

- Function call stack (recursion)
- Expression evaluation (infix, prefix, postfix)
- Undo / Redo operations
- Backtracking (DFS)
- Finding next/previous greater/smaller element
- Reversing data
- Browser history

“A stack helps you remember where you left!” 😊

“Last In, First Out (LIFO)”
Sometimes you have to go back to move forward! 😊

5 Important Patterns

- 1 Matching parentheses (valid expression)
- 2 Monotonic stack (next/previous greater/smaller)
- 3 Stack for expression conversion/evaluation
- 4 Using stack to reverse data
- 5 Stack with extra information (index, min, etc.)

6 Valid Parentheses - Example

Given a string s containing brackets "()", "{}", "[]", determine if it is valid.

```
bool isValid(string s) {
    stack<char> st;
    for(char c : s){
        if(c=="(" || c=="{" || c=="[" ) st.push(c);
        else{
            if(st.empty()) return false;
            char top = st.top(); st.pop();
            if((c==')' && top!='(') ||
               (c=='}' && top!='{') ||
               (c==']' && top!='[')) return false;
        }
    }
    return st.empty();
}
```

Example:

$s = "({[]})"$

Output: true

Note:

If stack is empty in between or mismatched bracket → return false.

7 Monotonic Stack - Next Greater Element

Maintain a decreasing stack to find next greater element.

```
vector<int> nextGreater(vector<int>& nums){
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st; // stores indices
    for(int i = 0; i < n; i++){
        while(!st.empty() && nums[i] > nums[st.top()]){
            res[st.top()] = nums[i];
            st.pop();
        }
        st.push(i);
    }
    return res;
}
```

Example:

$nums = [4, 5, 2, 10]$

Output:

$[5, 10, 10, -1]$

8 Must Solve Problems

#	Problem	Pattern	Difficulty
1	Valid Parentheses	Stack	Easy
2	Min Stack	Stack (design)	Medium
3	Next Greater Element I	Monotonic Stack	Medium
4	Daily Temperatures	Monotonic Stack	Medium
5	Largest Rectangle in Histogram	Monotonic Stack	Hard
6	Remove K Digits	Monotonic Stack (Greedy)	Hard

7

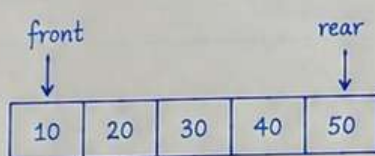
Queue & Deque

"Same people, different doors
Queue keeps it fair!"

1 What is a Queue ?

- A linear data structure that follows FIFO (First In, First Out) principle.
- Insertion happens at rear and deletion happens at front.
- Used in scheduling, BFS, caching, etc.
- Time Complexity: $O(1)$ for enqueue, dequeue, front (using linked list / circular array).

2 Queue Representation



Enqueue → Add at rear
Dequeue → Remove from front
Front → Get front element

3 Basic Operations (Code)

```
queue<int> q;
q.push(10); // [10]
q.push(20); // [10, 20]
q.push(30); // [10, 20, 30]

q.front(); // 10
q.pop(); // removes 10
q.size(); // 2
q.empty(); // false
```

In Java:

```
Queue<Integer> q =
    new LinkedList<>();
q.offer(10);
q.poll();
q.peek();
```

Note:

- In C++ use `<queue>`
- Queue is usually implemented using Linked List / Circular Array.

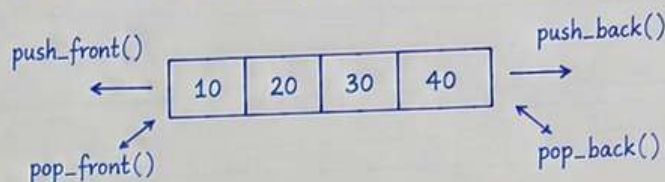
4 Types of Queues

- Simple Queue - standard FIFO
- Circular Queue - last position connected to first
- Deque (Double Ended Queue) - insert/delete from both ends
- Priority Queue - elements with priority (implemented using heap)
- Monotonic Queue - maintains elements in sorted order (used in sliding window maximum)

"Queues keep things in order,
even when life doesn't!" 😊

5 Deque (Double Ended Queue)

- Allows insertion and deletion from both front and rear.
- Useful for sliding window problems, maintaining window of elements, etc.
- In C++ → use `deque<>`
- In Java → use `ArrayDeque<>`



6 Deque Operations (Code)

```
deque<int> dq;
dq.push_back(10); // [10]
dq.push_back(20); // [10, 20]
dq.push_front(5); // [5, 10, 20]

dq.pop_back(); // [5, 10]
dq.pop_front(); // [10]
dq.front(); // 10
dq.back(); // 10
```

Note:

- All operations are $O(1)$.
- Very useful in sliding window maximum / minimum problems.

7 When to use Queue / Deque ?

Problem Type	Use
Level order traversal (BFS)	Queue
Shortest path (unweighted graph)	Queue
Scheduling / Task Management	Queue
Sliding window (max/min)	Deque (Monotonic)
LRU Cache	Deque + HashMap
0-1 BFS	Deque
Simulation problems	Queue

8 Must Solve Problems

#	Problem	Type	Difficulty
1	Implement Queue using Stacks	Queue	Easy
2	Shortest path (unweighted graph)	Queue	Easy
3	Design Circular Queue	Queue	Medium
4	Number of Islands (BFS)	Queue (BFS)	Medium
5	Rotting Oranges	Queue (BFS)	Medium
6	Sliding Window Maximum	Deque	Hard
7	Shortest Path in Binary Matrix	Queue (BFS)	Medium
8	LRU Cache	Deque + HashMap	Hard

8 Linked List

"Not about the nodes, but about the connections." 😊

1 What is a Linked List ?

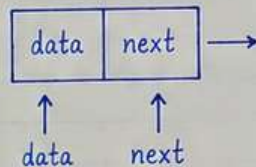
- A linear data structure where elements (nodes) are not stored at contiguous memory locations.
- Each node contains data and a reference (pointer) to the next node.
- Types : Singly, Doubly, Circular Linked List.
- Time Complexity: $O(1)$ for insertion/deletion (at head), $O(n)$ for traversal/search.

2 Node Structure

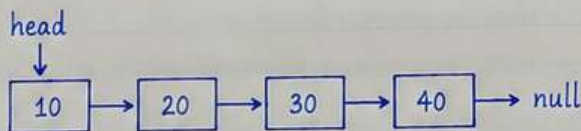
In C++ / Java (class)

```
class Node {
    int data;
    Node* next;
    Node(int val) {
        data = val;
        next = null;
    }
}
```

A Node



3 Singly Linked List Representation



4 Common Operations

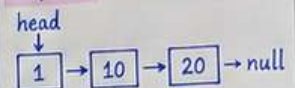
- 1 Traversal
- 2 Insertion (at head, tail, position)
- 3 Deletion (by value / position)
- 4 Search
- 5 Update
- 6 Reverse a linked list
- 7 Find middle (slow-fast)
- 8 Detect cycle
- 9 Merge two sorted lists
- 10 Reorder / manipulate

"Linked lists teach you to think in pointers!" 😊

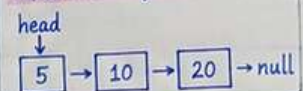
5 Insert at Head - Example

```
Node* insertAtHead(Node* head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;
    return newNode;
}
```

Before:



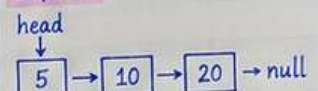
After inserting 5:



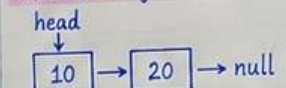
6 Delete a Node - Example

```
Node* deleteHead(Node* head) {
    if (head == null) return null;
    Node* temp = head;
    head = head->next;
    delete temp; // in C++
    return head;
}
```

Before:



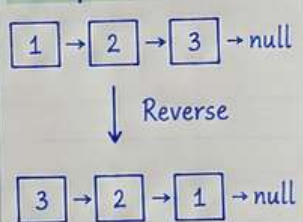
After deleting head:



7 Reverse a Linked List

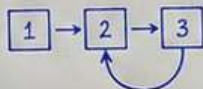
```
Node* reverseList(Node* head) {
    Node* prev = null;
    Node* curr = head;
    while (curr != null) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Example:



8 Detect Cycle (Floyd's Algorithm)

```
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != null && fast->next != null) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```



If slow == fast at any point → cycle exists.

9 Must Solve Problems

#	Problem	Pattern / Concept	Difficulty
1	Reverse Linked List	Iteration / Pointers	Easy
2	Merge Two Sorted Lists	Two Pointers	Easy
3	Linked List Cycle	Fast & Slow Pointer	Easy
4	Remove Nth Node From End	Two Pointers	Medium
5	Reorder List	Two Pointers	Medium
6	Copy List with Random Pointer	HashMap	Hard

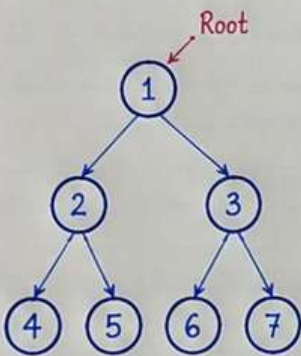
9 Trees - Basics

"A tree is not just a data structure, it's a way to organize possibilities."

1 What is a Tree ?

- A hierarchical data structure with a root node.
- Each node can have 0 or more children.
- No cycles (connected and acyclic).
- Common types: Binary Tree, Binary Search Tree, N-ary Tree, etc.
- Used in file systems, databases, parsers, etc.
- Time Complexity: $O(n)$ for most basic operations.

2 Tree Terminology



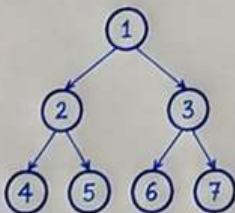
- Root: Topmost node
- Child: Direct descendant
- Parent: Node with children
- Leaf: Node with no children
- Height: No. of edges in longest path (root to leaf)
- Depth: No. of edges from root to node
- Subtree: A node and all its descendants

3 Tree Traversals

Type	Order	Use Case
Preorder	Root → Left → Right	Copy tree, prefix eval
Inorder	Left → Root → Right	BST gives sorted order
Postorder	Left → Right → Root	Delete tree, postfix eval
Level Order	Level by level (BFS)	Shortest path, levelwise operations

4 Traversal Examples

For this tree:



- Preorder: 1 2 4 5 3 6 7
- Inorder: 4 2 5 1 6 3 7
- Postorder: 4 5 2 6 7 3 1
- Level Order: 1 2 3 4 5 6 7

Note: Inorder traversal of a binary tree is sorted only if it is a BST.

"From roots to results!"



5 Height of a Tree

```
int height(Node* root) {
    if (root == nullptr) return 0;
    int leftH = height(root->left);
    int rightH = height(root->right);
    return 1 + max(leftH, rightH);
}
```

Time: $O(n)$
Space: $O(h)$
h = height of tree (recursion stack)

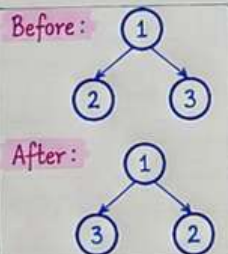
6 Check if Two Trees are Same

```
bool isSameTree(Node* p, Node* q) {
    if (p == nullptr && q == nullptr) return true;
    if (p == nullptr || q == nullptr) return false;
    if (p->val != q->val) return false;
    return isSameTree(p->left, q->left) &&
        isSameTree(p->right, q->right);
}
```

Time: $O(n)$
Space: $O(h)$

7 Invert a Binary Tree

```
Node* invertTree(Node* root) {
    if (root == nullptr) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```



8 Level Order Traversal (BFS)

```
vector<vector<int>> levelOrder(Node* root) {
    vector<vector<int>> res;
    if (root == nullptr) return res;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int size = q.size();
        vector<int> level;
        for (int i = 0; i < size; i++) {
            Node* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
```

Time: $O(n)$
Space: $O(n)$
(queue can hold up to $n/2$ nodes)

9 Common Problems (Basics)

#	Problem	Concept	Difficulty
1	Maximum Depth of Binary Tree	Height (DFS)	Easy
2	Same Tree	Recursion	Easy
3	Invert Binary Tree	Recursion	Easy
4	Level Order Traversal	BFS	Medium
5	Diameter of Binary Tree	DFS	Medium
6	Balanced Binary Tree	DFS (height)	Medium

10

Trees - Advanced

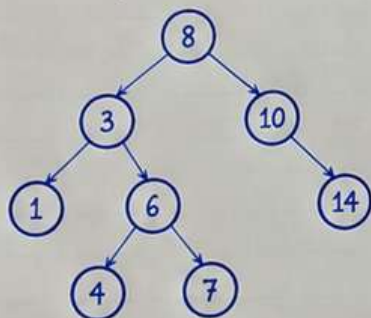
"Same data, different tree, different strengths."

1 Types of Trees (Quick View)

Type	Property	Use Case
Binary Tree	Each node has ≤ 2 children	General purpose
Binary Search Tree	Left < root < Right	Searching (sorted)
AVL Tree	Self-balanced (!height diff! ≤ 1)	Self-balancing BST
Heap (Min/Max)	Complete binary tree	Priority queue
Trie	Tree of characters	Autocomplete, dict.
Segment Tree	Range queries	Competitive programming
Fenwick Tree (BIT)	Prefix sums	Competitive programming
N-ary Tree	Each node has N children	File systems, org charts

2 Binary Search Tree (BST)

- Left subtree contains smaller values.
- Right subtree contains greater values.
- Inorder traversal gives sorted order.
- Time Complexity: Search/Insert/Delete $\rightarrow O(h)$
($O(\log n)$ for balanced, $O(n)$ for skewed)



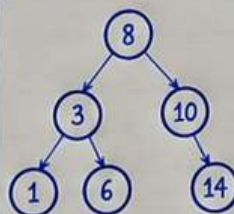
Inorder:

1, 3, 4, 6, 7, 8, 10, 14
(sorted)

3 Lowest Common Ancestor (LCA)

- Find the lowest node that has both nodes as descendants.
- Works efficiently for BST.

```
Node* lca(Node* root, int p, int q) {
    if (root == nullptr) return nullptr;
    if (p < root->val && q < root->val)
        return lca(root->left, p, q);
    if (p > root->val && q > root->val)
        return lca(root->right, p, q);
    return root;
}
```



LCA(1, 6) = 3
LCA(1, 14) = 8

4 Diameter of a Binary Tree

- Diameter = length of the longest path between any two nodes.
- Can pass through root or anywhere in the tree.
- Use DFS to calculate height and update diameter.

```
int diameter(Node* root, int &ans) {
    if (root == nullptr) return 0;
    int leftH = diameter(root->left, ans);
    int rightH = diameter(root->right, ans);
    ans = max(ans, leftH + rightH);
    return 1 + max(leftH, rightH);
}
```

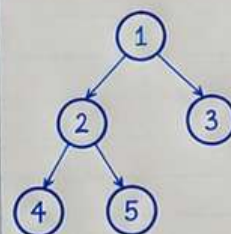
Time: $O(n)$
Space: $O(h)$
(recursion stack)

5 Serialize & Deserialize a Tree

- Convert a tree to a string (for storage / transmission).
- Use level order traversal with null markers.

Serialize (Level Order):

```
queue<Node*> q;
q.push(root);
while (!q.empty()) {
    Node* node = q.front(); q.pop();
    if (node) {
        s += to_string(node->val) + ',';
        q.push(node->left);
        q.push(node->right);
    } else {
        s += "#,";
    }
}
```



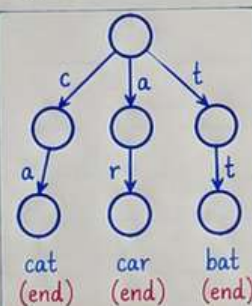
Serialized:

1,2,3,4,5,#,#,#,#

6 Trie (Prefix Tree)

- Used to store strings character by character.
- Useful for autocomplete, dictionary search, etc.

```
struct TrieNode {
    TrieNode* children[26];
    bool isEnd;
    TrieNode() {
        for (int i = 0; i < 26; i++)
            children[i] = nullptr;
        isEnd = false;
    }
};
```



7 Must Solve Problems

#	Problem	Type	Difficulty
1	Diameter of Binary Tree	DFS	Easy
2	LCA in Binary Tree	Tree	Medium
3	Serialize & Deserialize	Tree	Medium
4	Implement Trie	Trie	Medium
5	Word Search II (Trie)	Trie + Backtracking	Hard
6	Maximum Path Sum	Tree	Hard

"Good trees age well!"



11

Graphs - Basics

"Graphs connect the dots between problems and solutions."

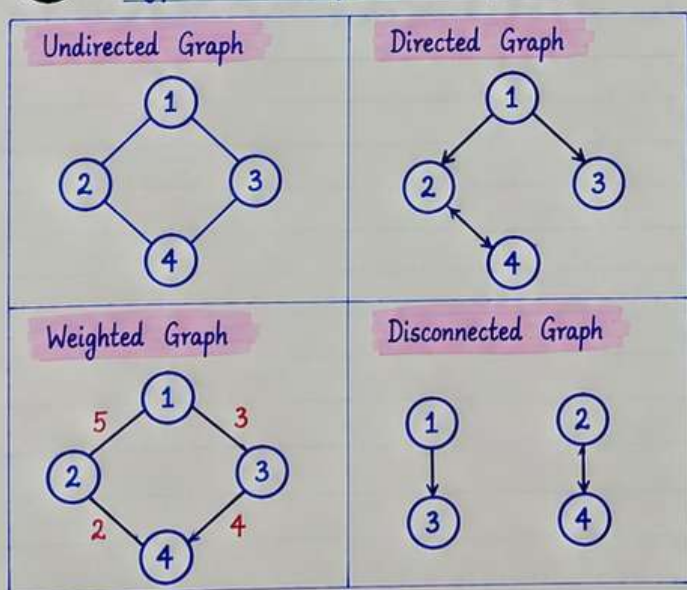
1 What is a Graph ?

- A graph is a non-linear data structure consisting of a set of nodes (vertices) and a set of edges that connect pairs of nodes.
- Used to represent real-world relationships (social networks, maps, networks, etc.).
- Can be directed or undirected, weighted or unweighted.
- Notation : $G = (V, E)$
- Time Complexity : Depends on the algorithm (e.g. BFS/DFS $\rightarrow O(V + E)$).

2 Graph Terminology

- Vertex (Node) : A point in graph.
- Edge : Connection between two vertices.
- Degree : Number of edges connected to a vertex.
- Path : Sequence of vertices connected by edges.
- Cycle : A path that starts and ends at the same vertex.
- Connected Graph : There is a path between every pair of vertices.
- Component : A maximal set of connected vertices.
- Weighted Graph : Edges have weights (cost/distance).
- Directed Graph (Digraph) : Edges have direction.
- Undirected Graph : Edges have no direction.

3 Types of Graphs (Examples)



"Graphs show us that everything is connected!" 😊

4 Graph Representations

1 Adjacency Matrix

- A $V \times V$ matrix where $mat[i][j] = 1$ (or weight) if there is an edge between i and j .
- Space : $O(V^2)$
- Good for dense graphs.

Example (Undirected)

$V = \{0, 1, 2, 3\}$	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	0
3	0	1	0	0

Note :
Matrix is symmetric for undirected graphs.

2 Adjacency List

- An array (or list) where each vertex stores a list of its adjacent vertices.
- Space : $O(V + E)$
- Good for sparse graphs.

Example (Undirected)

$V = \{0, 1, 2, 3\}$	0	$\rightarrow [1, 2]$
	1	$\rightarrow [0, 2, 3]$
	2	$\rightarrow [0, 1]$
	3	$\rightarrow [1]$

In C++ :
`vector<int> adj[V];`

In Java :
`List<List<Integer>>
adj = new
ArrayList<>();`

5 Common Graph Operations

- Add edge
- Remove edge
- Check if a path exists (BFS/DFS)
- Find connected components
- Detect cycle
- Find shortest path (BFS for unweighted, Dijkstra for weighted)
- Topological sort (for DAG)

6 Must Solve Problems

#	Problem	Concept	Difficulty
1	Number of Connected Components	BFS/DFS	Easy
2	Detect Cycle in Undirected Graph	DFS	Medium
3	Detect Cycle in Directed Graph	DFS (recursion stack)	Medium
4	Bipartite Graph Check	BFS (coloring)	Medium
5	Shortest Path in Unweighted Graph	BFS	Medium
6	Dijkstra's Algorithm	Greedy + PQ	Hard

12

Graphs - Advanced

"Graphs turn real-world chaos into solvable structure."

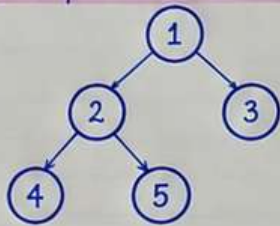
1 Graph Traversal - BFS (Code)

```
vector<int> bfs(int start, vector<vector<int>>& adj) {
    vector<int> vis(adj.size(), 0);
    queue<int> q;
    vector<int> order;
    q.push(start);
    vis[start] = 1;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        order.push_back(node);
        for (int nei : adj[node]) {
            if (!vis[nei]) {
                vis[nei] = 1;
                q.push(nei);
            }
        }
    }
    return order;
}
```

Time Complexity:
 $O(V + E)$
(adjacency list)

Space Complexity:
 $O(V)$

Example BFS Traversal



Start from 1:

Traversal Order:
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

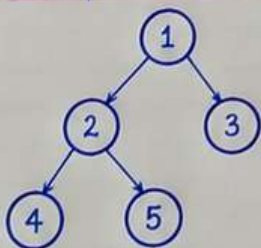
2 Graph Traversal - DFS (Code)

```
void dfs(int node, vector<vector<int>>& adj,
        vector<int>& vis, vector<int>& order) {
    vis[node] = 1;
    order.push_back(node);
    for (int nei : adj[node]) {
        if (!vis[nei]) {
            dfs(nei, adj, vis, order);
        }
    }
}
```

Time Complexity:
 $O(V + E)$
(adjacency list)

Space Complexity:
 $O(V)$
(recursion stack)

Example DFS Traversal



Start from 1:

One possible order:
 $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3$

Note:

- DFS order can vary based on the order of adjacency list.

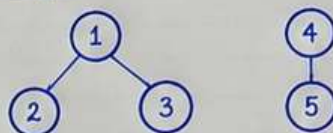
"Explore deeper to see the bigger picture."



3 Connected Components

- A connected component is a set of nodes where every node is reachable from every other node.
- In an undirected graph, run DFS/BFS from each unvisited node and count components.
- Useful in social networks, network connectivity, etc.
- Time Complexity: $O(V + E)$

Example:



Connected Components = 2

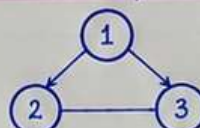
Note:

In directed graphs, we use Strongly Connected Components (SCC) (Kosaraju / Tarjan).

4 Cycle Detection

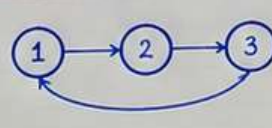
- Undirected Graph: Use DFS (if visited neighbor is not parent $\rightarrow$ cycle exists).
- Directed Graph: Use DFS + Recursion Stack or Kahn's Algorithm (for DAG check).
- Time Complexity: $O(V + E)$

Undirected Graph (DFS)



Cycle exists ✓

Directed Graph (Recursion Stack)

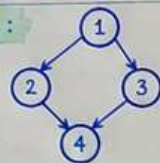


Cycle exists ✓

5 Topological Sort (DAG)

- A linear ordering of vertices such that for every edge $u \rightarrow v$, u comes before v .
- Applicable only in Directed Acyclic Graph (DAG).
- Used in task scheduling, course prerequisites, etc.
- Common Algorithms:
 - Kahn's Algorithm (BFS)
 - DFS (using stack)
- Time Complexity: $O(V + E)$

Example:



One Topological Order:
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

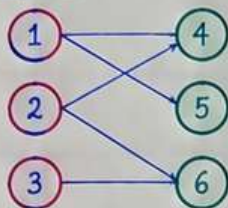
6 Must Solve Problems

#	Problem	Concept	Difficulty
1	Number of Connected Components	DFS	Easy
2	Cycle Detection (Undirected)	DFS	Easy
3	Cycle Detection (Directed)	DFS + Rec Stack	Medium
4	Topological Sort	BFS / DFS	Medium
5	Alien Dictionary	Topological Sort	Hard
6	Kosaraju's Algorithm (SCC)	Strongly Connected Components	Hard

1 Bipartite Graph

- A graph is bipartite if its vertices can be divided into two disjoint sets such that no two vertices in the same set are adjacent.
- No odd length cycles.
- Common use cases: Matching problems, scheduling, etc.
- Can be checked using BFS/DFS (2-coloring).

Example:



This is a bipartite graph.

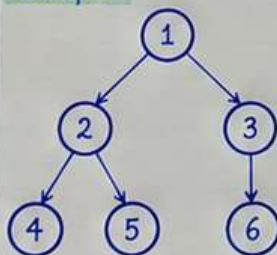
Note:

If we can color the graph using 2 colors such that no adjacent nodes have same color → Graph is bipartite.

2 Shortest Path - Unweighted Graph

- Use BFS to find shortest path (minimum number of edges) from a source to all other nodes.
- Time Complexity: $O(V + E)$

Example:



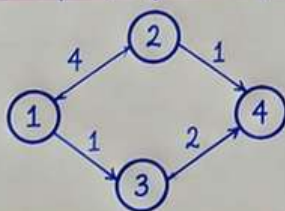
Shortest distance from 1:

Node	Distance
1	0
2	1
3	1
4	2
5	2
6	2

3 Shortest Path - Weighted Graph

- Use Dijkstra's Algorithm (for non-negative weights).
- Use Bellman-Ford (if negative weights exist).
- Time Complexity:
 - Dijkstra: $O((V + E) \log V)$
 - Bellman-Ford: $O(V \times E)$

Example (Weighted Graph):

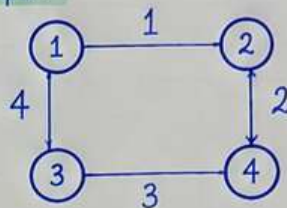


Shortest path from 1 to 4:
 $1 \rightarrow 3 \rightarrow 4$ (cost = 3)

4 Minimum Spanning Tree (MST)

- A spanning tree is a subset of edges that connects all vertices with minimum total weight.
- No cycles, exactly $(V - 1)$ edges.
- Common Algorithms:
 - Kruskal's Algorithm (uses Union-Find)
 - Prim's Algorithm (uses Priority Queue)
- Time Complexity: $O(E \log V)$

Example:



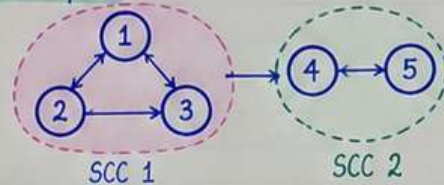
Note:

MST connects all nodes with minimum cost and no cycles. Useful in network design, roads, cables, etc.

5 Strongly Connected Components (SCC)

- In a directed graph, a strongly connected component is a group of nodes where every node is reachable from every other node.
- Common Algorithm: Kosaraju's Algorithm (or Tarjan's Algorithm)
- Time Complexity: $O(V + E)$

Example:



Note:

SCC helps in understanding cycles and component dependency in directed graphs.

6 Minimum Spanning Tree vs Shortest Path

Aspect	MST	Shortest Path
Goal	Connect all nodes (min cost)	Find minimum cost between nodes
Output	A tree ($V-1$ edges)	A path (or distances)
Common Algo	Kruskal / Prim	Dijkstra / Bellman-Ford
Use Case	Network design	Navigation, routing
Works On	Undirected graph	Directed / Undirected
Time Complexity	$O(E \log V)$	$O((V + E) \log V)$

7 Must Solve Problems

#	Problem	Concept	Difficulty
1	Is Graph Bipartite?	BFS (2-coloring)	Easy
2	Shortest Path in Unweighted Graph	BFS	Easy
3	Dijkstra's Algorithm	Shortest Path	Medium
4	Minimum Spanning Tree	Kruskal / Prim	Medium
5	Strongly Connected Components	Kosaraju / Tarjan	Hard
6	Network Delay Time	Shortest Path (PQ)	Medium

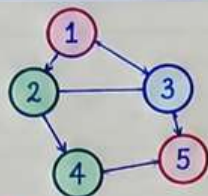
"Graphs turn complex problems into clear paths."



1 Graph Coloring

- Assign colors to vertices such that no two adjacent vertices have the same color.
- Applications: map coloring, exam scheduling, register allocation (compilers).
- Greedy Coloring (works well in practice).
- Time Complexity: $O(V + E)$

```
void graphColoring() {
    int color[V];
    fill(color, color+V, -1);
    color[0] = 0;
    for (int u = 1; u < V; u++) {
        bool used[V] = {false};
        for (int v : adj[u])
            if (color[v] != -1) used[color[v]] = true;
        int c = 0;
        while (used[c]) c++;
        color[u] = c;
    }
}
```

Example Coloring
(using 3 colors)

2 Detecting a Cycle (Directed Graph)

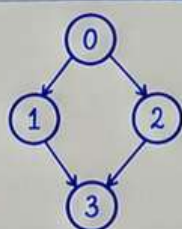
- Use DFS with a recursion stack.
- If we visit a node already in recursion stack $\rightarrow$ cycle exists.
- Time Complexity: $O(V + E)$

```
bool isCyclic(int v) {
    visited[v] = true;
    recStack[v] = true;
    for (int nei : adj[v]) {
        if (!visited[nei] && isCyclic(nei)) return true;
        else if (recStack[nei]) return true;
    }
    recStack[v] = false;
    return false;
}
```

3 Topological Sort (Kahn's Algorithm)

- Only for Directed Acyclic Graph (DAG).
- Use in task scheduling, course prerequisites.
- Steps: compute in-degree, push 0 in-degree nodes into queue.
- Time Complexity: $O(V + E)$

```
vector<int> topoSort(int V) {
    vector<int> indegree(V, 0), res;
    queue<int> q;
    for (int u = 0; u < V; u++)
        for (int v : adj[u]) indegree[v]++;
    for (int i = 0; i < V; i++)
        if (indegree[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        res.push_back(u);
        for (int v : adj[u])
            if (--indegree[v] == 0) q.push(v);
    }
    return res;
}
```

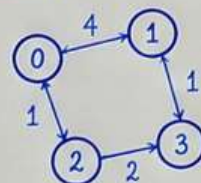
Topological Order:
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

4 Shortest Path - Dijkstra's Algorithm

- Find shortest path from a source to all other nodes in a weighted graph (non-negative weights).
- Use a min-heap (priority queue).
- Time Complexity: $O((V + E) \log V)$

```
vector<int> dijkstra(int src) {
    vector<int> dist(V, INT_MAX);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Example:

Shortest distances
from 0:

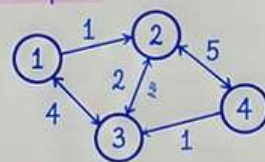
0: 0
1: 3
2: 1
3: 3

5 Minimum Spanning Tree - Kruskal's Algorithm

- Find a subset of edges that connects all vertices with minimum total weight.
- Use Disjoint Set Union (Union-Find).
- Time Complexity: $O(E \log E)$

```
sort(edges.begin(), edges.end());
for (auto [u, v, w] : edges) {
    if (find(u) != find(v)) {
        unionSet(u, v);
        mst_weight += w;
    }
}
```

Example:

MST weight = $1 + 1 + 2 = 4$

6 Real World Applications

- Social Networks: Friend recommendations, community detection.
- Maps & Navigation: Shortest path, route optimization.
- Web Crawling: Page ranking (Google).
- Network Design: Minimum Spanning Tree.
- Task Scheduling: Topological sort.
- Resource Allocation: Graph coloring.

7 Quick Summary

Algorithm / Concept	Graph Type	Time Complexity	Use Case
BFS	Unweighted	$O(V + E)$	Shortest path (unweighted)
DFS	Unweighted	$O(V + E)$	Cycle detection, traversal
Dijkstra	Weighted	$O((V + E) \log V)$	Shortest path (non-negative)
Kruskal	Undirected	$O(E \log E)$	Minimum Spanning Tree
Topological Sort	DAG	$O(V + E)$	Task scheduling
Graph Coloring	Any	$O(V + E)$	Resource allocation

15

Graphs - Recap & Practice

"Practice on graphs today,
better problem solving tomorrow."

1 Key Takeaways

- Graphs model real-world relationships.
- Can be directed/undirected, weighted/unweighted.
- Common representations: Adjacency Matrix, Adjacency List.
- Traversal algorithms: BFS, DFS.
- Important problems: shortest path, MST, cycle detection, topological sort, SCC, etc.
- Time complexity depends on graph size and algorithm.
- Graphs are used in networks, maps, social media, recommendation systems, and more.

"Choose the right graph + right algorithm
for the problem."



2 Applications in Real World

Domain	How Graphs are Used
Social Networks	Friend suggestions, community detection
Maps & Navigation	Shortest path, route optimization
Web (Google)	Page ranking (PageRank)
Recommendation	User-item relationships
Network Design	Minimum spanning tree
Task Scheduling	Topological sorting
Telecom / Internet	Routing, network connectivity
Games	Path finding (e.g. A* search)
Compilers	Dependency graphs
Other	Resource allocation, circuit design, etc.

3 Time & Space Complexity (Summary)

Algorithm	Time Complexity	Space Complexity
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
Dijkstra (Min Heap)	$O((V + E) \log V)$	$O(V)$
Bellman-Ford	$O(V \times E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal's (MST)	$O(E \log E)$	$O(V)$
Prim's (MST)	$O(E \log V)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V)$
Kosaraju (SCC)	$O(V + E)$	$O(V)$
Graph Coloring (Greedy)	$O(V + E)$	$O(V)$

"From nodes and edges
to endless possibilities."



4 Common Problem Patterns

Pattern	Typical Algorithm	Example Problem
Reachability	BFS / DFS	Is there a path between two nodes?
Shortest Path	BFS / Dijkstra	Find minimum distance between nodes.
Minimum Spanning Tree	Kruskal / Prim	Connect all nodes with minimum cost.
Cycle Detection	DFS / Union-Find	Does a cycle exist?
Topological Order	Kahn's Algorithm	Find build order / course schedule.
Strongly Connected Components	Kosaraju / Tarjan	Find all SCCs.
Graph Coloring	Greedy / Backtracking	Assign colors to vertices.
Bipartite Check	BFS / DFS	Can we divide into two sets?

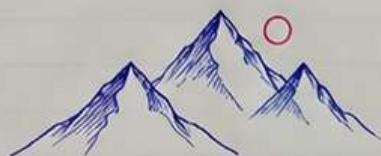
5 Practice Problems (Recommended)

#	Problem	Platform	Difficulty
1	Number of Connected Components	LeetCode 323	Easy
2	BFS of Graph	LeetCode 133	Medium
3	Clone Graph	LeetCode 133	Medium
4	Course Schedule (Topological Sort)	LeetCode 207	Medium
5	Course Schedule II	LeetCode 210	Medium
6	Pacific Atlantic Water Flow	LeetCode 417	Medium
7	Dijkstra's Algorithm	GFG / Implement	Medium
8	Network Delay Time	LeetCode 743	Medium
9	Minimum Spanning Tree	GFG / Implement	Hard
10	Strongly Connected Components	GFG / Implement	Hard

6 Next Steps

- Solve the recommended problems.
- Try variants and harder versions.
- Implement on your own (don't just copy).
- Apply graph concepts in real-world projects.
- Revise regularly and make your own templates.

"Graphs are not just a topic in DSA,
they are a way to see and solve real-world problems."



Keep
Learning
Keep Growing!

16

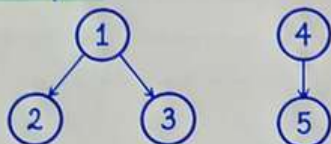
Graphs - Practice Problems

"Solve more problems, see the patterns, get better."

1 Connected Components

- Problem:** Given an undirected graph, find the number of connected components.

Example:



Output:
Number of components
= 2

2 Number of Islands (Grid)

- Problem:** Given a 2D grid of '1's (land) and '0's (water), find the number of islands.

Example:

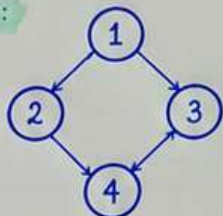
1	1	0	0	1
1	0	0	1	1
0	0	0	0	0
1	0	1	1	0

Output:
Number of islands
= 3

3 Cycle Detection in Undirected Graph

- Problem:** Given an undirected graph, determine if it contains a cycle.

Example:

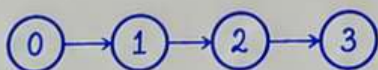


Output:
Cycle exists
= True

4 Course Schedule (Directed Graph)

- Problem:** Given numCourses and prerequisites, determine if you can finish all courses (i.e., no cycle in graph).

Example: numCourses = 4
prerequisites = [[1,0], [2,1], [3,2]]



Output:
Can finish all
courses = True

5 Word Ladder (Shortest Path)

- Problem:** Given a begin word, end word and a dictionary, find the length of the shortest transformation sequence (changing one letter at a time).

Example:

beginWord = "hit"
endWord = "cog"
wordList = ["hot", "dot", "dog", "lot", "log", "cog"]

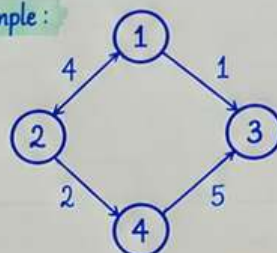
Output:

Shortest length
= 5
(hit → hot → dot → dog → cog)

6 Dijkstra's Algorithm (Weighted Graph)

- Problem:** Given a weighted graph and a source node, find the shortest distance to all other nodes.

Example:

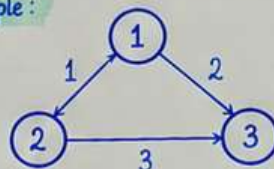


Output (source = 1):
Distance to 1 = 0
Distance to 2 = 4
Distance to 3 = 1
Distance to 4 = 6
(1 → 3 → 4)

7 Minimum Spanning Tree (Kruskal/Prim)

- Problem:** Given a connected, undirected, weighted graph, find the total weight of the Minimum Spanning Tree.

Example:



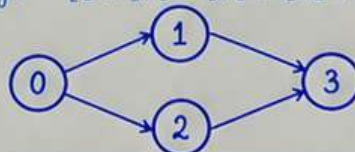
Output:
MST total weight
= 3
(edges: 1-2, 1-3)

8 Topological Sort (Directed Acyclic Graph)

- Problem:** Given a DAG, return a valid topological ordering of the nodes.

Example:

V = 4
edges = [[0,1], [0,2], [1,3], [2,3]]



Output:
One possible order
= [0, 1, 2, 3]

Keep Going

- Try variations of these problems.
- Solve on LeetCode / GFG and note patterns.
- Combine concepts (e.g., BFS + shortest path, DFS + cycle).
- Graphs are powerful - the more you practice, the clearer they get!

"Practice builds intuition, patterns make you faster."



17

Graphs - Practice Problems II

"More practice, stronger problem solving skills."

1 Number of Provinces (Connected Components)

- **Problem:** Given an adjacency matrix of an undirected graph, find the number of provinces (connected components).
- **Uses:** DFS / BFS.
- **Time Complexity:** $O(V + E)$

Example:

isConnected =

```
[[1, 1, 0],
 [1, 1, 0],
 [0, 0, 1]]
```

Output:

Number of provinces
= 2
(0 and 1 are connected,
2 is separate)

2 Rotting Oranges (Multi-Source BFS)

- **Problem:** Given a grid with fresh (1), rotten (2) and empty (0) cells, find the minimum time to rot all oranges.
- **Use:** Multi-source BFS (start from all rotten oranges).
- **Time Complexity:** $O(m \times n)$

Example:

grid =

```
[[2, 1, 1],
 [1, 1, 0],
 [0, 1, 1]]
```

Output:

Minimum time
= 4

3 Word Ladder (Shortest Transformation)

- **Problem:** Given beginWord, endWord and a dictionary, find the length of shortest transformation sequence (changing one letter at a time).
- **Use:** BFS (unweighted shortest path).
- **Time Complexity:** $O(N \times L^2)$ (N = word count, L = word length)

Example:

beginWord = "hit"
endWord = "cog"
wordList = ["hot", "dot",
"dog", "lot", "log", "cog"]

Output:

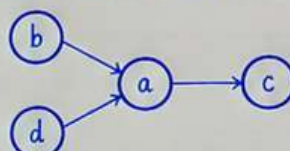
Shortest length
= 5
hit → hot → dot
→ dog → cog

4 Alien Dictionary (Topological Sort)

- **Problem:** Given a sorted dictionary of words from an alien language, find the order of characters.
- **Use:** Topological Sort (Kahn's Algorithm).
- **Time Complexity:** $O(V + E)$

Example:

words = ["baa", "abcd",
"abca", "cab", "cad"]



Output:

One possible order
= bdac
(there can be multiple
valid orders)

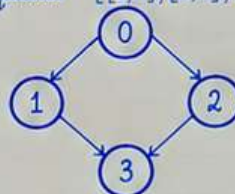
5 Course Schedule II (Find Order)

- **Problem:** Given numCourses and prerequisites, return a possible order to finish all courses.
- **Use:** Topological Sort (Kahn's Algorithm).
- **Time Complexity:** $O(V + E)$

Example:

numCourses = 4

prerequisites = [[1,0], [2,0], [3,1], [3,2]]



Output:

One possible order
= [0, 1, 2, 3]
(or [0, 2, 1, 3])

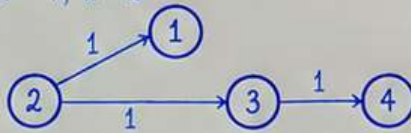
6 Network Delay Time (Dijkstra)

- **Problem:** Given a directed, weighted graph, find the time taken for a signal to reach all nodes from a given source.
- **Use:** Dijkstra's Algorithm (min-heap).
- **Time Complexity:** $O((V + E) \log V)$

Example:

times = [[2,1,1], [2,3,1], [3,4,1]]

n = 4, k = 2



Output:

Network delay time
= 2

Key Tips for Practice

- Identify graph type (directed/undirected, weighted/unweighted).
- Choose the right algorithm (BFS, DFS, Dijkstra, Topological Sort, etc.).
- Check for edge cases (disconnected graph, single node, cycles).
- Dry run on small examples before coding.
- Practice multiple problems on LeetCode / GFG.

"Same concepts,
different problems,
bigger thinking."

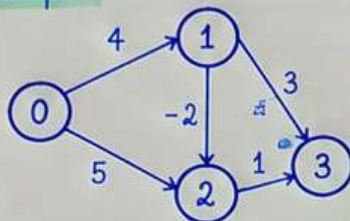


"It's not just about learning graphs, it's about thinking in connections."

1 Bellman-Ford Algorithm

- **Problem:** Find shortest path from a source node in a weighted graph (can have negative weights).
- **Use:** When graph contains negative edge weights.
- Works for both directed and undirected graphs.
- Detects negative weight cycles.
- Time Complexity: $O(V \times E)$

Example:



Output:

(from source 0):

$0 \rightarrow 0 = 0$
 $0 \rightarrow 1 = 4$
 $0 \rightarrow 2 = 2$
 $0 \rightarrow 3 = 3$

2 Floyd-Warshall Algorithm

- **Problem:** Find shortest paths between all pairs of nodes.
- **Use:** Works with negative weights (but no negative cycles).
- Dynamic Programming approach.
- Time Complexity: $O(V^3)$
- Space Complexity: $O(V^2)$

Example:

(Initial Matrix)

0	3	∞	7
∞	0	1	∞
∞	∞	0	2
∞	∞	∞	0

Output:

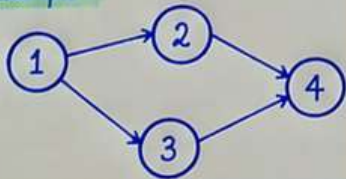
(Shortest Distance Matrix)

0	3	4	6
∞	0	1	3
∞	∞	0	2
∞	∞	∞	0

3 Kahn's Algorithm (Topological Sort)

- **Problem:** Find a topological ordering of a DAG.
- **Use:** Often used in task scheduling, course prerequisites.
- **Steps:**
 - ① Compute in-degree of all nodes.
 - ② Push all 0 in-degree nodes into a queue.
 - ③ Process nodes and reduce in-degree of neighbors.
 - ④ If all nodes are processed $\rightarrow$ valid topological order. If not $\rightarrow$ cycle exists.
- Time Complexity: $O(V + E)$

Example:



Output:

One possible order
 $= [1, 2, 3, 4]$

4 Disjoint Set Union (Union-Find)

- **Problem:** Keep track of disjoint sets and efficiently merge them.
- **Use:** Useful in Kruskal's algorithm and connectivity problems.
- **Operations:**
 - $\text{find}(x)$: find representative of set
 - $\text{union}(x, y)$: merge two sets
- **Optimizations:** Path Compression, Union by Rank.
- Time Complexity: $\sim O(\alpha(N))$ (almost constant).

Example:

$\text{union}(1, 2)$
 $\text{union}(2, 3)$
 $\text{find}(3) = 1$
 (1, 2, 3 are in same set)

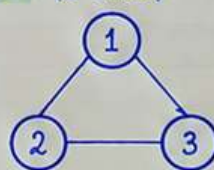
Output:

$\text{find}(3) = 1$
 Same component
 $\rightarrow \text{True}$

5 Graph Coloring (Backtracking)

- **Problem:** Assign colors to vertices such that no two adjacent vertices have the same color.
- **Use:** Map coloring, exam scheduling, register allocation.
- **Approach:** Try all possible colors using backtracking.
- Time Complexity: $O(m^V)$ (m = colors, V = vertices)

Example: (3 colors)



Output:

Possible coloring:
 $1 \rightarrow \text{Red}$
 $2 \rightarrow \text{Green}$
 $3 \rightarrow \text{Blue}$

6 Summary Table

Algorithm	Graph Type	Time Complexity	Key Use Case
BFS	Unweighted	$O(V + E)$	Shortest path (unweighted)
DFS	Unweighted	$O(V + E)$	Traversal, cycle detection
Dijkstra	Weighted	$O((V + E) \log V)$	Shortest path (non-negative)
Bellman-Ford	Weighted	$O(V \times E)$	Shortest path (with negative weights)
Floyd-Warshall	Weighted	$O(V^3)$	All pairs shortest path
Kruskal (MST)	Undirected	$O(E \log E)$	Minimum spanning tree
Prim (MST)	Undirected	$O(E \log V)$	Minimum spanning tree
Topological Sort	DAG	$O(V + E)$	Task scheduling
Graph Coloring	Any	$O(m^V)$	Resource allocation
DSU (Union-Find)	Any	$\sim O(\alpha(N))$	Connectivity, cycle check

"Practice. Visualize. Solve. Repeat.
 That's how graphs become second nature."



Keep Learning
 Keep Growing!